

- Summary
- Architecture
- How the Tensor API is used
- SAM 2 model graphs
- Frame model graphs
- ModelChain pipeline
- WebAssembly target
- Web demo
- Verification
- Performance
- Points for review
- Files and reproduction

SAM 2 video segmentation as one Tensor API pipeline, native and on WebGPU

Every computation in this demo is a model authored in C++ with the LiteRT Tensor API: frame preprocessing, the Hiera image encoder, per-object prompt and tracking steps with SAM 2's memory bank, and the image shown on screen. The models are wired together with LiteRT's `ModelChain`. The same C++ runs natively and, compiled to WebAssembly, in Chrome on WebGPU.

Model: SAM 2.1 Hiera-Tiny (`facebook/sam2.1-hiera-tiny`) Inputs: 384 / 512 / 1024 px
Measured on an M4 Pro Mac (48 GB), Chrome, Sept 2026

1 Summary

The demo is meant to show what the Tensor API can carry on its own: not only a model, but the whole application pipeline around it. Below are the claims, each with the evidence behind it. Section 11 lists the parts where the implementation departs from stock LiteRT usage and where review would help most.

Two models, authored entirely with the Tensor API, no converter.

verified

SAM 2 model: 11 signatures, 22 builtin op types, 164 MB. Frame model: 2 signatures, 15 builtin op types. No custom or composite ops.

Weights are shared across signatures in one flatbuffer.

verified

163.0 MB of unique constants; 50.5 MB referenced by 2+ signatures. Per-signature copies would total 401 MB.

The pipeline is LiteRT's ModelChain, unmodified.

verified

`tensor/runners/model_chain.cc` compiled as-is for both native (Bazel) and wasm (emscripten).

Both models run fully accelerated on LiteRT.js WebGPU.

verified

`CompiledModel.isFullyAccelerated` = true for the SAM 2 model (11/11 signatures) and the frame model (2/2).

Outputs match Hugging Face Sam2VideoModel.

verified

fp32 at 384, native CPU and WebGPU: 0 pixels different on every frame. Native CPU fp32 at 1024: mean IoU ≥ 0.9996 per object. fp16 (Metal, WebGPU): mean mask IoU 0.994–1.000 per object.

The browser can author the SAM 2 model itself.

verified

The wasm module reads `safetensors` weights and builds + serializes all 11 signatures in 0.2 s (M4 Pro); the result passes the same HF verification as the prebuilt model.

The browser pipeline keeps up with a live camera.

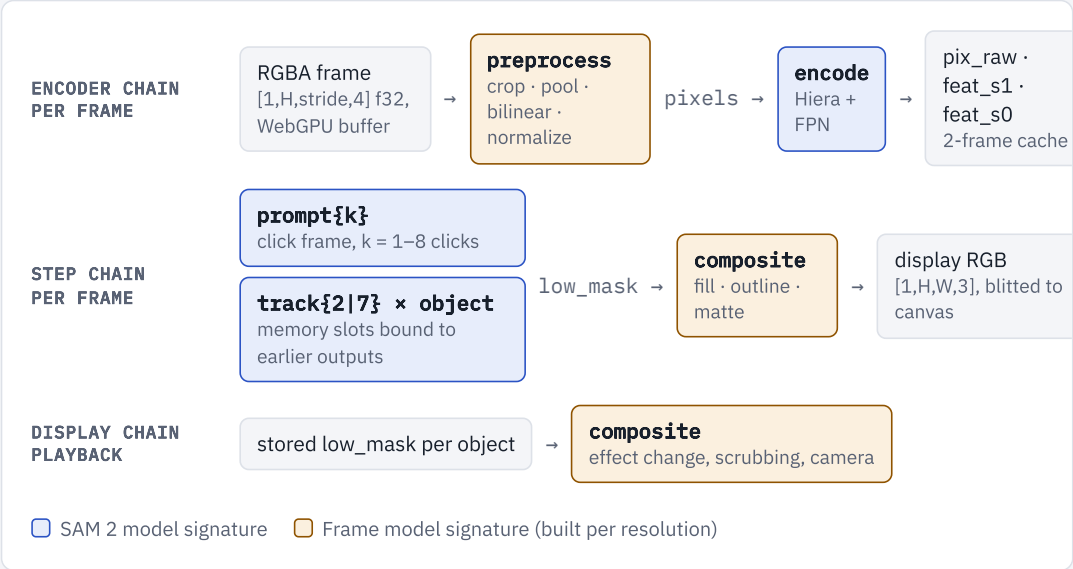
measured

M4 Pro, 1280×720 camera at 30 fps, one object: at 384 px, 30 fps with a 23 ms pipeline and 31 ms camera to screen; at 512 px, 29 fps with a 34 ms pipeline. The

display image is rendered in-graph.

2 Architecture

Two compiled models serve four kinds of `ModelChain`. The SAM 2 model is built once per input size. The frame model is built at run time for each video resolution, because preprocessing and compositing depend on the frame's exact width and height.



The memory bank is not a tensor the pipeline copies into. Each tracking step writes fresh `mem` and `ptr` buffers. The next frame's `track{n}` stage takes each memory slot and object pointer as its own input, and the host binds those inputs to the buffers earlier steps wrote. The concatenation into a bank happens inside the graph.

3 How the Tensor API is used

Everything that computes in this demo starts as C++ calls to the LiteRT Tensor API (`tensor/` in the LiteRT repository). This section describes the parts of the API the project relies on, how the network is expressed with them, and how the result becomes a `.tflite`. Code excerpts are verbatim from the sources; op counts are read from the serialized model.

Where the Tensor API code lives

The Tensor API code is spread over three places: this project's code, the SAM 2 network builders in `litert-samples`, and the Tensor API library in LiteRT. The native build (Bazel) and the wasm build (emscripten) compile the same files from all three.

LOCATION	FILE	ROI
This project cc/	chain_graphs.{h,cc}	The aut mo (e tr per anc (p co by
	sam2_pipeline.{h,cc}	The buf Ad aut

LOCATION	FILE	ROLE
		on cha
	signature_stage.{h,cc}	A t one Co
	sam2_chain_main.cc	Nat we Ad run
	host_plan.{h,cc}	Clic ten me
SAM 2 network builders litert-samples/models/sam2/ sam2_hiera_tiny_video/tensor_api/	sam2_image/sam2_graph.{h,cc}	Hie ma suc atte par
	sam2_video/sam2v_graph.{h,cc}	Me Rof the Bu SAI pro Bu gra wri me sar #3
	sam2_image/sam2_weights.cc, sam2_video/sam2v_weights.cc	Loa wei cor
Tensor API library LiteRT tensor/ (Bazel's cached copy of LiteRT)	tensor.h, arithmetic.h, internal/*	The op
	backends/tflite/tflite_flatbuffer_conversion.cc	Mo .t cor eag stu
	runners/model_chain.{h,cc}	Mo unr
Browser build wasm/	sam2_wasm.cc	The Jav Ad the (?l res (1: li is t Lite AP:

Tensors, graph inputs and constants

All graphs are built with one tensor type, `TfTensor` = `litert::tensor::Tensor<TfLiteMixinTag>`. The mixin tag selects the TFLite backend: every op call registers a `graph::Operation` through `TfLiteMixinRegistrar`, which knows how to emit it as a TFLite builtin. A tensor is created from a `TensorInit {name, type, shape, buffer}`:

- **No buffer: a graph input.** `Input("mem_3", {1, 576, 64})` declares a placeholder. Its name becomes the signature's input name.
- **With an `OwningCpuBuffer` : a constant.** Every weight and every baked table is a constant tensor holding its fp32 data.
- **Ops are free functions returning new tensors** (`FullyConnected` , `BatchMatMul` , `Conv2D` , `Softmax` , ...). Shapes are inferred as the graph is built, so builder code can read `x.GetShape()` to derive later shapes. Every graph in the demo has fully static shapes.

Model blocks are ordinary C++ functions composed from these ops. Three from the sample show the style:

`sam2_image/sam2_graph.cc`, `sam2_video/sam2v_graph.cc` (litert-samples)

```
// LayerNorm over the last axis: MEAN, SUB, MUL, MEAN, ADD, RSQRT, MUL, MUL, ADD
TfTensor LayerNormRaw(const TfTensor& x, const TfTensor& weight, const TfTensor& bias) {
    int last = static_cast<int>(x.GetShape().size()) - 1;
    TfTensor mean = Mean(x, {last}, /*keep_dims=*/true);
    TfTensor centered = Sub(x, mean);
    TfTensor var = Mean(Mul(centered, centered), {last}, /*keep_dims=*/true);
    TfTensor noimed = Mul(centered, Rsqrt(Add(var, ConstScalar(eps))));
    return Add(Mul(noimed, weight), bias);
}

// Attention core, rank 4 [B,H,N,D]: BATCH_MATMUL, SOFTMAX, BATCH_MATMUL.
// q arrives pre-scaled: 1/sqrt(d) is folded into the q projection weights.
TfTensor AttentionRaw(const TfTensor& q, const TfTensor& k, const TfTensor& v) {
    TfTensor scores = BatchMatMul(q, k, /*adj_x=*/false, /*adj_y=*/true);
    TfTensor attn = Softmax(scores);
    return BatchMatMul(attn, v);
}

// RoPE with precomputed tables; the rotate-half sign is baked into sin (SLICE, CONCAT)
TfTensor Rope(const TfTensor& x, const TfTensor& cos, const TfTensor& sin) {
    return Add(Mul(x, cos), Mul(SwapHalf(x), sin));
}
```

The sample's builders also carry optional `StableHLOComposite` forms (`odml.layer_norm` , `odml.scaled_dot_product_attention`) and an `odml.runtime_bmm` attention form behind config flags. They were evaluated earlier and not adopted: the shipped models use the raw decompositions above and contain no composite or custom ops.

Weights

- **Loading.** A Python export converts `facebook/sam2.1-hiera-tiny` to safetensors with weights already in TFLite layouts (`FULLY_CONNECTED [out, in]` , convolutions `OHWI`, depthwise `[1, kh, kw, C]` , RoPE-permuted q/k). `LoadCheckpointWeights` reads it into a `WeightMap` of name $\rightarrow$ constant `TfTensor` (F16/BF16 widened to fp32). Builders fetch weights by their Hugging Face names: `W(weights, "trunk.blocks.3.attn.qkv.weight")` .
- **Resolution tables.** The per-size tables (Hiera position embedding, RoPE cos/sin, sine position encodings) are entries in the same file. Where a table must be broadcast, the builder reads it back to host floats and re-bakes it as a new constant at the broadcast shape, because a graph `Reshape` of a constant was rejected by the GPU delegate.

From graph to .tflite: ModelFactory

- **One signature per** `AddSignature(inputs, outputs, key)` . `ModelFactory` explores backwards from the outputs to build one subgraph. The named placeholders become the signature's ordered inputs.
- **Constants are shared by identity.** Buffers are keyed by the tensor's buffer object across all subgraphs. The same weight `TfTensor` used by 11 signatures is serialized once. A table re-baked inside a builder would be a new buffer on every call, which is why the

- builders cache baked constants in `ConstCache` . Measured result: 163.0 MB of unique constants, against 401 MB if every signature had its own copy.
- **Serialization.** `Save(path)` writes the flatbuffer and the buffer data (with `XNN_EXTRA_BYTES` tail padding). The same call writes to the in-memory filesystem in the wasm build.

Architecture to emitted ops

Because the graphs are hand-written, each component maps to a predictable set of TFLite ops. The counts below are predicted from the builders and checked against the serialized 384 px model; every one matches.

COMPONENT	TENSOR API CONSTRUCTION	OPS (PREDICTED = EMITTED)
Patch embed	<code>PadHW(3) + Conv2D 7×7 / stride 4, VALID + Add(pos_embed)</code>	<code>PAD, CONV_2D</code> ×1
Hiera trunk, 12 blocks (depths 1-2-7-2)	per block: <code>2 LayerNormRaw · FullyConnected qkv · Mha · FullyConnected-Gelu-FullyConnected MLP; MaxPool2D query pooling at 3 stage transitions (shortcut + query)</code>	<code>RSQRT 24 (12×2) · SOFTMAX 12 · BATCH_MATMUL 24 · GELU 12 · MAX_POOL_2D 6 (3×2)</code>
FPN neck + decoder skips	<code>4 lateral Conv2D 1×1 · ResizeNearestNeighbor top-down · conv_s0 / conv_s1</code>	<code>CONV_2D 6 (7 in encode) · RESIZE_NEAREST_NEIGHBOR 1</code>
Memory attention, 4 layers (track only)	per layer: self-attention with Rope on q, k · cross-attention to the memory with Rope on q and spatial keys, key_mask added to scores · ReLU MLP (FullyConnected with fused activation) · 3 LayerNorms; final LayerNorm	<code>BATCH_MATMUL 16 (4×2×2) · SOFTMAX 8 · RSQRT 13 (4×3+1)</code>
Mask decoder	two-way transformer: <code>2 blocks × 3 attentions + final token-to-image attention · hypernetwork mask product · output upscaling 2× TransposeConv + LayerNorm + 2 Gelu · SamMlp3 heads (IoU with sigmoid, object score, object pointer)</code>	<code>BATCH_MATMUL 15 (7×2+1) · SOFTMAX 7 · TRANSPOSE_CONV 2</code>
Memory encoder	<code>mask downsampler 4 × (PadHW + Conv2D 3×3 / 2 + LayerNorm + Gelu) + 1×1 · pix_feat_proj · 2 ConvNext fusers (DepthwiseConv2D 7×7) · output projection</code>	<code>CONV_2D 7 · DEPTHWISE_CONV_2D 2</code>
Totals per step	<code>prompt1 = decoder + BuildStep post + memory encoder; track7 = memory attention + the same</code>	<code>prompt1: BATCH_MATMUL 18 (15 + 3 selection), SOFTMAX 7, RSQRT 16, GELU 8 · track7: BATCH_MATMUL 34, SOFTMAX 15, RSQRT 29</code>

The step signatures' LayerNorm count of 16 is the decoder's 9 (4 per two-way block plus one final) plus the upscaling LayerNorm plus the memory encoder's 6 (4 downsampler plus 2 fuser). Their 8 GELUs are the upscaling's 2, the downsampler's 4 and the fusers' 2.

What this project writes with the API

- `BuildStep` (section 4; written for this demo, now in the sample): SAM 2's per-frame host logic as ops (`StepPos` one-hot selection applied with `BatchMatMul` , `Mean` stability areas, arithmetic blends, `ResizeBilinear` upsampling).
- **The signatures** (section 4): `prompt1..8` with `[1, k+1, 256]` sparse inputs; `track2 / track7` whose memory bank is a `Concatenation` of named per-slot placeholders.

- **The frame model** (section 5): preprocessing and the display composite, authored per video resolution at run time.

The Tensor API in the browser

The Tensor API library (`tensor.cc` , `buffer.cc` , `internal/*` , the TFLite backend) is compiled to wasm from the same sources as the native build. Two things run in the page as a result:

- **On every geometry change** (a new video or the camera), the page authors, serializes and compiles a new frame model for that exact resolution.
- **With** `?build=browser` , the page authors the full SAM 2 model from safetensors weights: the same `AddSam2Signatures` , 11 signatures, 5,550 ops, in 0.2 s.

The only part of the TFLite backend excluded from wasm is the eager `Run()` helper, which embeds a TFLite interpreter; execution goes through `CompiledModel` instead (section 7).

4 SAM 2 model graphs

The network graphs come from the `litert-samples SAM 2 Tensor API example` (`models/sam2/sam2_hiera_tiny_video/tensor_api`): Hiera encoder, memory attention, mask decoder, memory encoder. This project defines the signature set below on the sample's fused step graph, `BuildStep` . The model is serialized by `ModelFactory` into one flatbuffer.

Signatures (384 px model)

SIGNATURE	OPS	INPUTS	OUTPUTS	ORIGIN
encode	521	pixels [1,384,384,3]	pix_raw [1,24,24,256], feat_s1 [1,48,48,64], feat_s0 [1,96,96,32]	sample
prompt1	420	pix_raw, feat_s1, feat_s0, nomem [1,1,1,1], sparse [1,2,256]	mem [1,576,64], ptr [1,256], low_mask [1,96,96], object_score [1,1], iou [1,1]	added
prompt2 ... prompt8	449	same, sparse [1,k+1,256]		added
track2	733	pix_raw, feat_s1, feat_s0, nomem, mem_0..1 [1,576,64], ptr_0..15 [1,256], slot_tpe [1,2,1,64], ptr_pos [1,1,64,64], key_mask [1,1,1,1216]	same five outputs	added
track7	733	same with mem_0..6; 30 inputs; key_mask [1,1,1,4096]		added

Op set across all signatures (22 types): ADD, BATCH_MATMUL, CONCATENATION, CONV_2D, DEPTHWISE_CONV_2D, FULLY_CONNECTED, GELU, LOGISTIC, MAX_POOL_2D, MEAN, MUL, PAD, RELU, RESHAPE, RESIZE_BILINEAR, RESIZE_NEAREST_NEIGHBOR, RSQRT, SLICE, SOFTMAX, SUB, TRANSPOSE, TRANSPOSE_CONV. Attention is written as raw ops (BATCH_MATMUL + SOFTMAX) with the $1/\sqrt{d}$ scale folded into the query weights, LayerNorm as MEAN/RSQRT, and RoPE in memory attention with precomputed cos/sin tables whose sin carries the rotate-half sign, so there is no NEG. The step signatures contain no BOOL tensors.

The tracking step: memory slots as separate inputs

The upstream memory-attention builder takes a single `mem_bank [1,N,HW,64]` placeholder. Here that placeholder is replaced by a concatenation of per-slot inputs before the builder runs, so the signature exposes one input per stored frame.

cc/chain_graphs.cc – AddSam2Signatures

```
for (int n : {2, 7}) {
  s2v::MemCondInputs mc = s2v::MakeMemCondInputs(config, n);
  std::vector<TfTensor> mems, ptrs;
  for (int k = 0; k < n; ++k)
    mems.push_back(Input(MemInput(k), {1, hw, kMemCh})); // "mem_k"
  for (int k = 0; k < kNumPtrFrames; ++k) // 16 object p
    ptrs.push_back(Input(PtrInput(k), {1, kHidden})); // "ptr_k"
  mc.mem_bank = Reshape(Concatenation(absl::MakeSpan(mems), /*axis=*/1), {1, n, hw,
  mc.ptr_tok = Reshape(Concatenation(absl::MakeSpan(ptrs), /*axis=*/1),
    {1, 1, kNumPtrFrames * kPtrSplit, kMemCh});

  s2v::VideoDecoderInputs dec = s2v::MakeVideoDecoderInputs(config);
  dec.pix_feat = s2v::BuildMemCond(config, n, mc, weights, &cache); // memory atte
  dec.sparse = Const(track_sparse, {1, 2, kHidden}); // baked "no p
  s2v::StepOutputs out = s2v::BuildStep(config, dec, mc.pix_raw, weights,
    /*multimask=*/true, /*binarize_mode=*/0, &ca
  // inputs: pix_raw, feat_s1, feat_s0, nomem, mem_0..n-1, ptr_0..15, slot_tpe, ptr_
  AddSig(factory, ins, out.AsList(), TrackSignature(n));
}
```

Unused slots are bound to a zero buffer and masked out by `key_mask (-30000, fp16-safe)`. Slot order does not change the result: every memory frame's keys carry the same spatial RoPE positions, and attention over keys is permutation-invariant given the mask.

BuildStep: SAM 2's per-frame host logic moved into the graph

`BuildStep` (in the sample's `sam2v_graph.cc`) turns decode → host post-processing → memorize into one subgraph. Thresholds use `StepPos(x) = 1 - Relu(1 - s·Relu(x))`, which equals $x > 0$ except within $1/s$ of 0 ($s = 10^4$):

- **Mask choice.** One click or a tracked frame: best of mask tokens 1–3 by predicted IoU, first maximum wins, as a one-hot built from pairwise `StepPos( $s_i - s_j + 10^{-5}$ )` and applied to the candidate masks, pointers and scores with a rank-4 `BatchMatMul`. Two or more clicks: token 0 unless its stability score (area of logits $> 0.05 \div \text{area} > -0.05$) is below 0.98, matching HF's `dynamic_multimask_via_stability`. Areas use `Mean`, not `Sum`: a pixel count overflows fp16 at 1024.
- **No object.** `StepPos(object_score)` gates a blend to mask `-1024`, pointer `no_obj_ptr` and occlusion flag 1, with no branch.
- **4× upsample for the memory encoder.** One `ResizeBilinear` with `align_corners=False`.
- **Memory input.** `mask_for_mem = 20·binarize(mask) - 10` in the prompt signatures and `20·sigmoid(mask) - 10` in the track signatures, then the memory encoder.

Graph choices made for the GPU delegates. These are deliberate departures from the most direct formulation, and a reviewer may know better alternatives:

- `nomem` (1 on the click frame, 0 otherwise) is a graph input, not a baked constant. As a constant it produced constant-constant MUL/SUB nodes the delegate rejected.
- Thresholds and gates are `StepPos` (Relu, Mul, Sub) instead of `Greater / GreaterEqual / Cast(bool) / ReduceMax`, so each step signature stays in one WebGPU delegate partition with no CPU fallback.
- Baked decoder tables (dense positional encoding, token tables) are cached across signatures (`ConstCache`), so 11 signatures share one copy.

Why one prompt signature per click count

SAM 2 encodes k prompt points as k point tokens plus one "not a point" pad token. Each token is the point's Fourier position encoding plus a learned embedding for its label: 1 positive, 0 negative, and 2 / 3 for a box's top-left / bottom-right corner. As in Hugging Face's video processor, a box is these two corner points placed before any clicks, so boxes (and box + clicks) run through the same `prompt{k}` signatures; the host fills the sparse rows. Padding to a fixed count is not neutral: the extra tokens take part in the decoder's token-to-image attention. So there are eight signatures, `prompt1 ... prompt8`, with sparse inputs `[1, k+1, 256]`. The shared weights make each extra signature cheap.

384 and 512 px from the 1024 px weights

No learned weight depends on input size. The export re-derives only the resolution-dependent tables (Hiera absolute position embedding, memory-attention RoPE grid, sine position encodings) using Hugging Face's own model configured at that size. It first checks that the same procedure at 1024 reproduces the original tables bit-exactly. The graph builders are already parameterized by input size.

5 Frame model graphs

`Sam2Pipeline::SetGeometry(w, h)` authors a second model for the video's exact resolution, serializes it, and compiles it through the same `CompiledModel` path. In the browser this happens inside the `wasm` module. At 640×360 it has 2 signatures and 81 ops.

preprocess (4 ops at 640×360)

`cc/chain_graphs.cc` — `AddFrameSignatures`

```
TfTensor frame = Input("frame", {1, H, geo.stride, 4}); // RGBA in [0,1], rows p
TfTensor rgb = Slice(frame, {0, 0, 0, 0}, {1, H, W, 3});
if (pool.ky > 1 || pool.kx > 1) // integer downscale fac
    rgb = AveragePool2D(rgb, pool.ky, pool.kx, pool.ky, pool.kx, kPaddingValid);
TfTensor resized = ResizeBilinear(rgb, {S, S}, /*align_corners=*/false, /*half_pixel
TfTensor pixels = Add(Mul(resized, Const(1/std, {3})), Const(-mean/std, {3}));
```

The frame tensor's row stride is padded to a multiple of 16 pixels, so the browser can fill it with a single `copyTextureToBuffer` from an `rgba32float` texture (256-byte rows). The graph crops the padding. The average pool is a simple anti-alias before large downscales. It is not the same filter as Hugging Face's video processor, which is why parity is measured on the pipeline's own preprocessed frames (section 9).

composite (77 ops)

This graph reproduces the original demo's WebGL mask renderer. It takes the frame, up to six objects' low-resolution logits, and an 8-float effect vector.

1. Stack the five masks, `ResizeBilinear` to H×W (SAM 2's post-processing, `align_corners=False`).
2. Signed distance to the boundary in display pixels: $d = 1 / \max(|\nabla I|, 1e-3)$. Central differences are two constant `DEPTHWISE_CONV_2D` over an edge-replicated border (`Slice` + `Concatenation`). Zero padding drew a false outline along the frame edge in `fp16`. d is clamped to ± 1000 so an absent object (logits -1024) cannot overflow `fp16`.
3. Overlay: per object, fill $0.5 \cdot \text{clamp}(d + \frac{1}{2})$ under a white ring $0.95 \cdot \text{clamp}(\text{stroke}/2 + \frac{1}{2} - |d|)$, composited premultiplied "over" in object order.

4. Matte: union coverage `ReduceMax` cuts the objects out over a dimmed grey frame (spotlight; a 3×3 `FULLY_CONNECTED` colour matrix) or green (cutout).
5. The effect vector blends overlay and matte arithmetically, so one graph serves all three effects.

6 The ModelChain pipeline

One compiled model, many stages

`CompiledModelStage` owns its `CompiledModel`. Creating one per stage would compile the 164 MB model and upload its weights up to about 20 times (encode, prompt1..8, and a track stage per object). `CompiledModel`'s non-owning constructor is protected, so the project adds `SignatureStage`: a `ModelStage` over a `shared_ptr<CompiledModel>` and a signature key. Descriptor discovery copies `CompiledModelStage`'s logic, including its buffer-type preference.

cc/signature_stage.cc — Run

```
absl::Status SignatureStage::Run() {
  std::vector<litert::TensorBuffer> ins, outs;
  for (const auto& n : input_names_) {
    auto dup = inputs_.at(n)->tensor_buffer().Duplicate();
    ins.push_back(std::move(*dup));
  }
  for (const auto& n : output_names_) {
    ...
    outs.push_back(std::move(*outputs_.at(n)->tensor_buffer().Duplicate()));
  }
  return model_->Run(signature_index_, ins, outs);
}
```

Chains

CHAIN	STAGES AND CONNECTIONS	BUILT
Encoder	<code>preprocess.pixels</code> → <code>encode.pixels</code>	per geometry
Step	<code>obj{k}_prompt{c}</code> or <code>obj{k}_track{n}</code> (per active object), each <code>.low_mask</code> → <code>composite.mask_k</code>	per object configuration, cached
Display	<code>composite</code>	per geometry

cc/sam2_pipeline.cc — StepChain

```
ModelChain::Builder b;
b.WithEnvironment(env_);
for (int k = 0; k < kMaxObjects; ++k) {
  if (steps[k].empty()) continue;
  const std::string name = absl::StrCat("obj", k, "_", steps[k]); // e.g. obj1_track1
  c->steps[k] = SignatureStage::Create(name, sam2_, steps[k]); // shared model
  b.AddStage(c->steps[k]);
  b.Connect(name, "low_mask", "composite", MaskInput(k));
}
b.AddStage(c->composite);
c->chain = std::make_unique<ModelChain>(*b.Build());
```

A tracked frame

For each object with a prompt before frame `t`, the host computes the memory plan (which stored frames fill which slots, plus the small temporal tables) and binds buffers. The whole frame is then one `ModelChain::Execute()`.

cc/sam2_pipeline.cc — Track

```

const MemPlan plan = PlanMemory(consts_, o.cond_frame, t, nmm_, hw_, has_mem, has_ptr);
BindEncoded(s, t); // pix_raw / feat_s1 / feat_s2
s.SetInputBuffer("nomem", zero_);
for (int i = 0; i < nmm_; ++i) // memory bank = buffer binding
    s.SetInputBuffer(MemInput(i), i < plan.slot_frames.size() ? o.mem.at(plan.slot_frames[i]) : zero_);
for (int i = 0; i < kNumPtrFrames; ++i)
    s.SetInputBuffer(PtrInput(i), i < plan.ptr_frames.size() ? o.ptr.at(plan.ptr_frames[i]) : zero_);
WriteFloats(*s.GetInputBuffer("slot_tpe"), plan.slot_tpe);
WriteFloats(*s.GetInputBuffer("ptr_pos"), plan.ptr_pos);
WriteFloats(*s.GetInputBuffer("key_mask"), plan.key_mask);
BindOutputs(s, k, t); // fresh mem / ptr / low_mask
...
c->chain->Execute(); // all objects' steps, then call

```

`BindOutputs` gives each step fresh output buffers from a small pool, so frame `t`'s memory survives for later frames. The same `low_mask` buffer is bound to the composite's matching input, which replaces the intermediate buffer `ModelChain::Build` allocated for that connection (see review point 2). Memory is evicted outside the 6 most recent frames plus the prompt frame; pointers outside the last 15. The host plan is plain C++ with 7 gtests against Hugging Face goldens.

7 The WebAssembly target

Why the runtime underneath is LiteRT.js

Open-source LiteRT cannot currently compile its WebGPU runtime to `wasm`. The `ml_drift` GPU accelerator source is private (its `http_archive` has no URL). The published WebGPU accelerator binaries target Android, Linux and Windows only. The Tensor API's WebGPU backend and runner (`tensor/backends/webgpu` , `tensor/runners/webgpu` , used by `tensor/wasm/demo/segmentation_webgpu_wasm.cc`) are internal-only. LiteRT.js's own `wasm` is a closed `embind` module with no C API exports and no dynamic linking.

So everything above the LiteRT C API is compiled to `wasm` as-is: the Tensor API, `ModelFactory` serialization, the SAM 2 graph builders, `ModelChain` , the LiteRT C++ API headers, and the pipeline. Only the runtime below that API is replaced.

A browser runtime behind the LiteRT C API

The C++ API reaches the runtime through one function table, `LiteRtRuntimeCapiStruct` (ABI 1.1.0), obtained from `GetLiteRtRuntimeBuiltin()` when no runtime is passed. The `wasm` build defines that function:

- **65 of 169 entries implemented.** The rest are generated stubs that return `kLiteRtStatusErrorUnsupported` and log their name (`gen_runtime_stubs.py` parses the header), so a missing entry is visible at once instead of being a null call.
- **Model introspection in C++.** Signatures, tensor names, shapes and types are read from the `.tflite` with the flatbuffers schema. No JS round trip.
- **Compile, run and buffers in JS** (`litert_js_bridge.js` , suspending through JSPI):
`loadAndCompile` on WebGPU; `run` wraps the input `GPUBuffer` s as `Tensor` s; buffers are `GPUBuffer` s on LiteRT.js's device; lock-for-read is a staging copy plus `mapAsync` ; unlock-after-write is `queue.writeBuffer` .
- **Requirements.** Every port reports `kLiteRtTensorBufferTypeWebGpuBuffer` , so `ModelChain` negotiates WebGPU buffers and intermediates never leave the GPU.

`wasm/litert_js_bridge.js` - `lrtjs_run` (abridged)

```

for (let i = 0; i < nIn; i++) { // no copies: wrap the chain'
    let t = byShape.get(d.shapeKey); // one Tensor per buffer and
    if (!t) t = new core.Tensor(lrt.buffers.get(bufId), d.shape, d.dtype);
    inputs[name] = t;
}

```

```

}
out = await entry.model.run(key, inputs); // LiteRT.js on WebGPU
const enc = device.createCommandEncoder();
for (let i = 0; i < nOut; i++) // LiteRT.js allocates output
  enc.copyBufferToBuffer(out[outNames[i]].toGpuBuffer(), 0, dst, 0, bytes);
device.queue.submit([enc.finish()]);

```

Build

- emscripten 6.0.10 with CMake. It uses the same LiteRT snapshot, absl and flatbuffers as the native Bazel build (the `litert-samples` workspace's external repositories), so both targets compile identical sources.
- `-sJSPI` for synchronous C++ over async WebGPU, and `-fwasm-exceptions`. With JS-emulated exceptions, the `invoke_*` wrappers become suspending imports and `std::locale`'s static constructor fails at load.
- One upstream file is patched at build time: `tflite_flatbuffer_conversion.cc`. Its `eager Run()` helper builds a TFLite interpreter with XNNPACK and is stubbed; `ModelFactory` is untouched. `XNN_EXTRA_BYTES` is defined as 16, its value outside Hexagon.
- `litert/build_common/build_config.h` is generated empty, as the native build does. `litert_logging.cc` and `litert_layout.cc` are compiled because the C++ headers call them directly.
- Output: `sam2_chain.wasm` 1.77 MB, `sam2_chain.mjs` 111 KB.

Authoring the model in the page

With `?build=browser`, the page downloads the safetensors weights instead of a `.tflite`, and the wasm module runs the same `AddSam2Signatures` as the native build. That is 11 signatures and 5,550 ops, serialized by `ModelFactory` to the in-memory filesystem in 0.2 s (M4 Pro), then compiled by LiteRT.js. It passes the same verification as the prebuilt model (section 9).

8 The web demo

- **Frames in, without CPU pixels.** `copyExternalImageToTexture` (video frame or `ImageBitmap`) into an `rgba32float` texture, then `copyTextureToBuffer` straight into the pipeline's `frame` tensor.
- **Picture out, without readback.** The composite output buffer is drawn to a WebGPU canvas by a 20-line blit shader. Click markers are a 2D canvas on top. There is no other rendering code.
- **Model loading.** Model files (100–200 MB) are kept in the browser's Cache API and reused while a HEAD request reports the same ETag, Last-Modified and size. A download that is not a TFLite flatbuffer (no `TFL3` identifier) is rejected before it reaches the wasm parser.
- **Serialization.** The C++ pipeline is one stateful object, so the UI queues all calls. Scrubbing requests are coalesced to the latest frame.
- **Camera.** Smooth (default): every camera frame goes through the display chain with the newest masks, between pipeline steps. Video runs at camera rate; masks trail by about one step. Aligned: each processed frame with its own masks. The input-size switch is disabled while the camera runs. `?profile=gpu|cpu` times each stage; `?cam=WxH` requests a camera size.
- **Ask Gemma.** Text to boxes: Gemma 4 (E4B / E2B) on LiteRT-LM, in a small OpenAI-compatible server on the LiteRT-LM Python API on the same machine (language model and vision encoder on the GPU), receives the frame on screen as a JPEG with Gemma's

native detection prompt and streams back `box_2d = [ymin, xmin, ymax, xmax]` in 0–1000; each box becomes a SAM 2 box prompt as soon as it is written. Gemma runs outside the page because LiteRT-LM's web Gemma 4 builds are text-only today. On the sample, "the soccer ball" with E4B gives the whole ball; "all players" fills up to five object slots; the first object appears after ~3 s with E4B (~1.5 s with E2B), then about one per second.

- **Boxes.** The Box tool: drag a rectangle around the object, in file or camera mode. Clicks can then refine it; a new box replaces the previous one. Up to 8 points per object, a box counting as 2.
- **Camera clicks.** Positive and negative clicks accumulate on the selected object (up to 8). Each click re-prompts the object with all its clicks on the newest camera frame, which becomes its prompt frame; Reset clears it. Every pipeline call, including clearing an object, goes through the same queue, so nothing mutates the pipeline during a step.
- **UI.** The same as the earlier LiteRT.js demo: sample video, upload, up to 6 objects, 8 positive and negative clicks each, 2- or 7-frame memory, 384/512/1024 px, overlay, spotlight and green cutout.

9 Verification

Method. The pipeline runs on the football sample (640×360). The rows below were measured with three objects: 2 clicks; 1 click; 3 clicks including a negative, joining at frame 6. `tools/verify_all.sh` now fills all five object slots (2 clicks; 1 click; a box plus a negative click, joining at frame 6; a box; a box joining at frame 3) and passes the same checks on an M3 (every object vs HF: mean IoU 0.98–1.00, WebGPU fp32 exact); a C++ unit test checks box encoding against Hugging Face's prompt encoder. The verifier then checks three things:

1. The `preprocess` output against a numpy implementation of the same graph (TFLite resize semantics).
2. Every object's mask on every frame against Hugging Face `Sam2VideoModel`, one streaming session per object, same prompts and memory size. The reference is computed once from the numpy preprocess reference and cached, so native and browser runs face the same ground truth.
3. The `composite` output against a numpy implementation.

RUN	FRAMES	OBJ 0	OBJ 1	OBJ 2	PREPROCESS	COMPOSITE
Native CPU fp32, 384, 7-frame, overlay	10	1.0000	1.0000	1.0000	7e-5	1e-5
Native CPU fp32, 1024, 7-frame	24	0.9996	1.0000	0.9999	1e-6	7e-6
Native Metal fp16, 384, 2-frame, cutout	10	0.996	0.999	0.997	4e-3	3e-4
WebGPU fp16 (wasm), 384, 7-frame, overlay	10	0.998	1.000	0.994	3e-3	6e-4
WebGPU fp16, 384, model authored in page, 2-frame, cutout	10	0.999	1.000	0.994	3e-3	2e-4
WebGPU fp32 (wasm), 384, 7-frame, overlay	10	1.0000	1.0000	1.0000	7e-5	1e-5

Object columns are mean mask IoU against Hugging Face. Rows at 384 are from `tools/verify_all.sh 384` on the M4 Pro; the 1024 row is a separate native run. Both 384 fp32 rows had 0 pixels different on every frame, so the fp16 differences are rounding, not the WebGPU path; at 1024, fp32 differs by at most 7 pixels on any frame (object 0) and 1 pixel

(object 2). The graph rewrites change fp32 rounding order (for example, the attention scale is folded into the weights), which is the likely cause; these few-pixel differences were not traced individually. The preprocess and composite columns give the maximum and the 99.9th-percentile absolute difference respectively, in normalized units and in display colour 0–1. Every run also passes a per-frame bound: IoU ≥ 0.95 (fp32) or ≥ 0.85 (fp16), or at most 3 pixels different.

The larger differences traced to SAM 2's hard decisions.

- Best-of-3 by predicted IoU: at one frame, HF's candidates 1 and 3 scored 0.8967 and 0.8958. fp32 rounding in LiteRT picked the other one, and the gap re-converged after 3 frames through memory.
- The 0.98 stability switch for 2+ clicks: the first whole-player prompt tried had stability 0.9817, and WebGPU fp16 landed below the threshold. The test prompt now has a 0.011 margin. The knife-edge case is kept as documentation, not hidden.

End-to-end UI check (Chrome, WebGPU):

- positive clicks grow the mask, a negative click shrinks it, and Reset clears it;
- two objects tracked over all 192 frames, with masks on every frame and no identity switch (frames where a mask collapses under occlusion are excluded from the jump test: 2 for the player, 10 for the ball);
- all three effects render;
- no page scroll at 1440, 1280 and 390 px;
- camera, 24 fps test camera: Smooth shows video at 24 fps with masks 53 ms behind; Aligned runs at 24.2 fps, 32 ms camera to screen (real-camera numbers are in section 10).

WebGPU health check (Chrome, Apple GPU, Metal-3, `shader-f16` , not a fallback adapter):

- every signature of both models fully on WebGPU;
- two objects over 192 frames: median 26 ms per frame, p95 27 ms, the same in the last quarter as the first;
- GPU buffers flat across repeated tracking and playback (1,240 → 1,284 → 1,284), freed by Reset (1,284 → 778, 41.4 → 35.6 MB), flat in camera mode (143);
- 0 WebGPU errors, no device loss.

Not measured: agreement with Hugging Face when HF applies its own video processor to the raw frames. The processor needs torchvision, which was not installed. The pipeline's preprocessing is verified against its own specification, and the masks against HF on those same frames.

10 **Performance**

Measured in Chrome on an M4 Pro Mac (48 GB), WebGPU fp16, with a live 1280×720 camera at 30 fps (33.3 ms per frame), one object and 2-frame memory. The demo's panel splits the latency of a displayed frame into Delivery (camera frame to the page), Wait (for the previous step to finish) and Pipeline (upload, preprocess, encode, track, composite, until the GPU is done); Camera → screen is their sum.

INPUT	DISPLAY	RATE	CAMERA → SCREEN	DELIVERY	WAIT	PIPELINE
384	Smooth	video 30 fps, masks 30.1 fps (lag 37 ms)	3 ms*	3 ms	0 ms	27 ms

INPUT	DISPLAY	RATE	CAMERA → SCREEN	DELIVERY	WAIT	PIPELINE
384	Aligned	30.4 fps	31 ms	8 ms	0 ms	23 ms
512	Smooth	video 26 fps, masks 26.0 fps (lag 68 ms)	36 ms*	5 ms	24 ms	38 ms
512	Aligned	29.1 fps	62 ms	5 ms	23 ms	34 ms

* Smooth: camera frame to the display composite being submitted; the masks trail by the lag shown.

GPU time per stage (?profile=gpu , a GPU sync after each stage, so the total is a little longer than the pipelined step; medians of 60 frames, ms):

INPUT	UPLOAD	PREPROCESS	ENCODE	TRACK2	COMPOSITE	C++ / JSPI	READBAC
384	1.0	0.7	12.2	7.7	3.5	0.5	0.
512	0.7	0.6	20.0	12.0	3.7	0.3	0.
1024	0.6	1.1	91.0	81.7	3.7	0.3	0.

- **The live rate is bound by GPU time.** CPU dispatch is small (512, ?profile=cpu : encode 0.2 ms, track2 0.5 ms, composite 0.1 ms).
- **384 fits in the camera interval**, so Wait is 0 and Aligned shows each frame 31 ms after capture. At 512 the step is just over 33 ms, so each frame waits about one camera interval. Keeping a second frame in flight does not help: the wait moves into Pipeline.
- **Encode and track dominate:** 75% of the frame at 384, 85% at 512. Composite, upload and preprocess depend on the camera size (?cam=WxH), not the model size.

Not measured here: 7-frame memory, several objects, and native Metal timings.

11 Points for review

These are the places where the implementation makes a choice a LiteRT expert is best placed to judge.

1 SignatureStage instead of CompiledModelStage.

Needed only because CompiledModelStage owns its model and CompiledModel's non-owning constructor is protected. A CompiledModelStage::Create(shared_ptr<CompiledModel>, signature) upstream would remove this class. Is sharing one CompiledModel across stages supported, including its Run being called for different signatures in sequence?

2 Rebinding ModelChain's connection buffers every frame.

The pipeline calls SetOutputBuffer and SetInputBuffer on stages after Build() , replacing the intermediate buffer allocated for low_mask → composite with a per-frame buffer, so masks and memory outlive the frame. It works because stages read their binding maps at Run() . Is this an intended use of ModelChain, or should per-execution output retention be a ModelChain feature?

3 Native intermediates on Metal.

The stage copies ModelChain's preference, which picks host memory whenever an accelerator lists it. Natively on Metal this may place intermediates in host buffers, with copies between stages; this was not profiled. In wasm every port reports WebGPU buffers only.

4 One GPU copy per output in the browser.

LiteRT.js's `run()` allocates its own outputs, so the bridge copies each into the chain's buffer (about 2.4 MB per frame). An output-binding `run` in LiteRT.js would make the browser path fully zero-copy.

5 Replacing `GetLiteRtRuntimeBuiltin()`.

The wasm runtime implements 65 entries of the internal `LiteRtRuntimeCapiStruct` (ABI 1.1.0). Is that table the right seam for an alternative runtime, and how stable is its ABI?

6 Patching `tflite_flatbuffer_conversion.cc`.

Only the eager `Run()` helper is stubbed. Splitting that helper into its own target upstream would let `ModelFactory` build for wasm without a patch.

7 Delegate-driven graph forms.

`nomem` as an input, `StepPos` arithmetic instead of comparisons and `Cast(bool)`, `Mean` instead of `Sum` for fp16 range, `-30000` mask fill. Are these still needed with current delegates, and is there a preferred idiom?

8 Hard thresholds under fp16.

SAM 2's argmax and 0.98-threshold decisions are exact in fp32 and can flip in fp16 when inputs sit on the boundary. Running just the decoder's scoring ops in fp32 on WebGPU, if a per-op precision control exists, would remove this.

9 Requirements.

Chrome with JSPI (on by default since Chrome 137) and native wasm exceptions.

12 Files and reproduction

FILE	LINES	ROLE
cc/chain_graphs.{h,cc}	398	Tensor API graphs: SAM 2 signatures, frame model
cc/signature_stage.{h,cc}	286	ModelStage over a shared CompiledModel
cc/sam2_pipeline.{h,cc}	686	ModelChains, bindings, memory, retention
cc/host_plan.{h,cc}, host_plan_test.cc	387	Click / pointer encodings, memory plan, 7 gtests
cc/sam2_chain_main.cc	330	Native driver: author model, run clip, dump
wasm/litert_js_runtime.cc	670	LiteRT runtime table backed by LiteRT.js
wasm/litert_js_bridge.js	220	compile / run / buffers on WebGPU (JSPI)
wasm/sam2_wasm.cc	278	embind API
app/src/chain/runtime.ts, app/src/display.ts	366	Load and model cache, frame upload, blit

FILE	LINES	ROLE
tools/verify_chain.py, verify_all.sh, inspect_model.py	—	Verification and model inventory

```
tools/verify_all.sh 384      # A: C++ unit + native CPU/Metal · B: wasm build + WebGF
wasm/build.sh          # emscripten build → app/public/wasm
cd app && npm run dev      # http://localhost:5175  (?size=512|1024, ?nmm=7, ?builc
python tools/inspect_model.py model.tflite  # signatures, ops, weight sharing
```

Prepared from this sample's sources and measured runs. Model graphs for the encoder, memory attention, decoder, memory encoder and `BuildStep` come from the `litert-samples SAM 2 Tensor API` example (`BuildStep` and the graph rewrites were merged there in PRs #356–#358); the prompt/track signatures, the frame model, the pipeline, and the wasm runtime are this project's.